A Tutorial on using the BlackCat Debugger with Catalina

BASIC CONCEPTS **3**
What is BlackCat? 3
What is the status of BlackCat? 3
What are the prerequisites for BlackCat? 3
How do I install BlackCat? 4
Who developed BlackCat? 4
Are there any known problems with BlackCat? 4
Who should I contact about BlackCat? 4
Why is it called BlackCat? 4
PREPARING PROGRAMS FOR USE WITH BLACKCAT **4**
Overview 4
Preparing a program using Catalina from the command line 5
Preparing a program using Catalina from Code::Blocks 6
Loading a program into the Propeller 6
Starting BlackCat 6
USING BLACKCAT **8**
Overview 8
The BlackCat Window *8*
The Propeller Communications Window *9*
Connecting to the Propeller 10
Optionally, select the Address Mode 10
BlackCat Menu Bar Commands 11
Open dbg file *11*
Clear Output Panel *11*
Options *11*
Show Virtual Breakpoints 11
Show Traffic in Communications Window 11
Remove all Breakpoints 11
Show Comms Stats *11*
Go to "stopped at" line *12*
Examine Memory *12*
Trace *12*
Watch *12*
Help *12*
Displaying files in the Source Window 12
Using Bookmarks 12
Managing the Variable "Show" list or Read/Write Variables 13
Displaying and Editing Variables 13
AN EXAMPLE OF USING BLACKCAT **14**
Overview 14
Compile the example program 14
Loading the example program 15
Starting BlackCat, establishing comms and loading the dbg file 15
Using Step, Step Into & Step Out 17
User Breakpoints 18
Viewing and Modifying Variables 19
KNOWN ISSUES WITH BLACKCAT **21**
ADVANCED TOPICS **21**
Debug files and listing files 21

Modifying Kernel and Hub / XMM memory locations.....21
Modifying or Recompiling programs.....22

Basic Concepts

This document provides a brief tutorial to get you up and running with the BlackCat source level debugger for the Catalina C compiler. It starts out with some basic concepts, but also introduces some of the more advanced concepts – however, it is not necessary to complete the whole tutorial to begin using BlackCat.

What is BlackCat?

BlackCat is a Windows application that allows source level debugging of C programs that have been compiled using the Catalina C compiler, and are executing on a supported Propeller platform.

BlackCat runs on a host PC, and uses a serial communications link to communicate with the program executing on the Propeller.

BlackCat provides the ability to display C source files, set and clear user breakpoints, single step through C programs, and display or modify the values of the local or global C variables visible whenever the program is stopped.

What is the status of BlackCat?

BlackCat is currently in full release. BlackCat is capable of debugging arbitrary C programs, including all the example programs provided with Catalina.

BlackCat supports CMM, LMM and NMM programs on all platforms supported by Catalina, and both SMALL and LARGE mode XMM programs on some of those platforms – see the prerequisites section below.

What are the prerequisites for BlackCat?

BlackCat runs only under Windows. It has been tested under Windows XP SP2 (32 bit) & Windows 7 (64 bit) but should also run on other versions of Windows. For Linux users, there is a command-line alternative called Black*Box* – see the **BlackBox Reference Manual** for details.

Programs to be debugged using this version of BlackCat must be compiled using Catalina release 4.0 or greater – BlackCat may not work with any earlier versions of Catalina. Programs must be prepared by compiling them using the **-g** command line option to Catalina (the meaning and use of this option is described later in this document).

BlackCat requires that the program being debugged has a cog free to implement the BlackCat serial protocol. Other than that, the overhead of BlackCat on the compiled program is limited to the execution of approximately 100 additional instructions (400 bytes) during program initialization. When not actually stopped at a breakpoint, the program executes as usual (i.e. at normal speed) and is unaware of BlackCat – this makes BlackCat suitable for debugging real-time or time-critical programs.

Catalina itself can be used under either Linux or Windows – i.e. programs to be debugged using BlackCat under Windows can be compiled using Catalina under Linux. However, for the sake of brevity, this document assumes you are using both BlackCat and Catalina under Windows.

BlackCat also requires a USB or RS232 serial connection to the Propeller running the program to be debugged.

Copyright 2013 Ross Higson

How do I install BlackCat?

BlackCat will be installed as part of each Catalina release. This document assumes you have installed Catalina in the default location (under Windows this is *C:\Program Files (x86)\Catalina)*.

If you have already installed Catalina, but to a directory other than the default directory, then all the Catalina commands and options shown will be identical – but some of the directory names will need to be changed.

Who developed BlackCat?

BlackCat was developed co-operatively by Bob Anderson and Ross Higson.

Bob Anderson is responsible for the overall design of BlackCat, the parsing of debug information generated by Catalina into a form suitable for use with BlackCat, the BlackCat user interface and the BlackCat debug cog that interacts with the Catalina Kernel.

Ross Higson is responsible for Catalina. Changes required to Catalina to support BlackCat include a modified Catalina code generator that emits debugging information during compilation, BlackCat specific targets that incorporate BlackCat-specific program initialization code.

Are there any known problems with BlackCat?

Yes, there are a few things not yet implemented, and a few other issues – see the section **Known Issues** later in this document.

Who should I contact about BlackCat?

Contact Ross Higson at ross@thevastydeep.com

Why is it called BlackCat?

During World War II, a Catalina aircraft used for combat missions – particularly night missions – was often painted matt black and known as a “Black Cat”.

See http://www.daveswarbirds.com/blackcat

Preparing programs for use with BlackCat

Overview

Preparing a program can be as simple as including the **–g** or **–g3** command line options when compiling the program with Catalina. This instructs Catalina to do two things:

1. Generate additional information that is required by the BlackCat debugger. A **.debug** file is generated for each C source file and these are then combined into one **.dbg** file that can be loaded by BlackCat.
2. Select a BlackCat enabled target file for the program. BlackCat targets are provided for CMM, LMM, EMM and XMM programs. These target files contain additional initialization code that loads the BlackCat debug cog, and performs other additional tasks required to prepare the kernel for co-operation with the debug cog.

In addition, the **–g3** option instructs Catalina to disable some compiler optimizations that may interfere with debugging - for example, it ensures that all functions have a stack frame whether they need it or not. However, it can increase program size and execution time slightly. In most cases, **-g3** is recommended, unless it makes the program too large to load, or execute too slowly.

There are a few things that you need to be aware of when preparing programs for use with BlackCat:

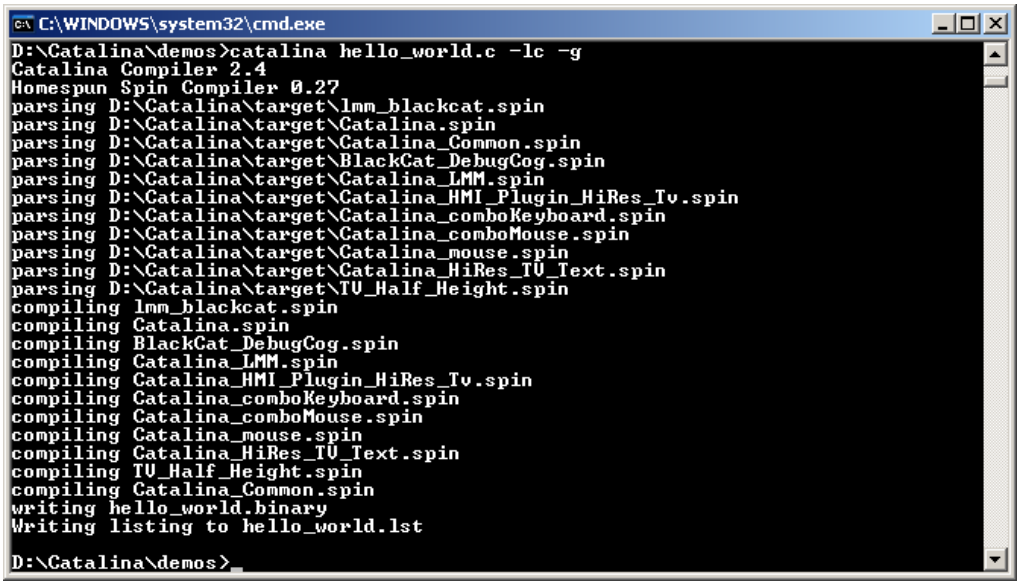
- The program must have a spare cog available.
- The program must have two Propeller pins to be used for serial communication with the BlackCat debugger running on the host (normally this is pins 30 & 31).
- You cannot specify your own target – to use BlackCat you must use the default target, which allows Catalina to select a BlackCat-enabled target file.
- The resulting program executable will be slightly larger – about 400 additional bytes of initialization code, plus up to 512 bytes of debug cog code (however, the space used for the debug cog code is reclaimed during initialization for use as program stack or heap space once the program is executing).

Preparing a program using Catalina from the command line

As mentioned above, simply include the **–g** or **–g3** command line options along with all the other normal options when compiling the program. For example, after setting up to use Catalina (using the **use_catalina** batch file) execute the following command in the Catalina demo directory:

```
catalina hello_world.c -lc -g3
```

This will produce output similar to the following:



```
C:\WINDOWS\system32\cmd.exe
D:\Catalina\demos>catalina hello_world.c -lc -g
Catalina Compiler 2.4
Homespun Spin Compiler 0.27
parsing D:\Catalina\target\lmm_blackcat.spin
parsing D:\Catalina\target\Catalina.spin
parsing D:\Catalina\target\Catalina_Common.spin
parsing D:\Catalina\target\BlackCat_DebugCog.spin
parsing D:\Catalina\target\Catalina_LMM.spin
parsing D:\Catalina\target\Catalina_HMI_Plugin_HiRes_Tv.spin
parsing D:\Catalina\target\Catalina_comboKeyboard.spin
parsing D:\Catalina\target\Catalina_comboMouse.spin
parsing D:\Catalina\target\Catalina_mouse.spin
parsing D:\Catalina\target\Catalina_HiRes_TV_Text.spin
parsing D:\Catalina\target\TV_Half_Height.spin
compiling lmm_blackcat.spin
compiling Catalina.spin
compiling BlackCat_DebugCog.spin
compiling Catalina_LMM.spin
compiling Catalina_HMI_Plugin_HiRes_Tv.spin
compiling Catalina_comboKeyboard.spin
compiling Catalina_comboMouse.spin
compiling Catalina_mouse.spin
compiling Catalina_HiRes_TV_Text.spin
compiling TV_Half_Height.spin
compiling Catalina_Common.spin
writing hello_world.binary
Writing listing to hello_world.lst

D:\Catalina\demos>_
```

Note that Catalina has automatically used the **lmm_blackcat.spin** target (instead of the more usual **lmm_default.spin** target).

In addition to the normal **hello_world.binary**, Catalina will also generate the following files:

- A file called **hello_world.lst** (a listing file is *always* produced when the **–g** command-line option is included).
- A file called **hello_world.debug** (one such file is generated file for each C source file, named with the same name as the C source file but with a **.debug** extension)
- A file called **hello_world.dbg** (one such file is generated for the whole compilation, named with the same name as the output binary file but with a **.dbg** extension).

Preparing a program using Catalina from Code::Blocks

The **Code::Blocks** "Debug" Build targets are automatically configured specify the **–g** option required for using BlackCat, while the "Release" Build targets will not.

To select which version to build, use the menu command **Build->Select a Target**. The Release version will be built in a *bin\Release* directory and the debug version will be built a *bin\Debug* directory.

Note that the **.debug** files generated by Catalina will be placed in the same directory as the source files, but the **.lst** and **.dbg** files will be placed in the same directory as the binary output file. This can be important if you want to manually load the **.dbg** file into BlackCat (this is described later).

Loading a program into the Propeller

BlackCat currently provides no support for loading programs into the Propeller. You simply use the normal Catalina Payload loader. All programs built using the **–g** or **–g3** options always stop after the execution of the initialization code, and wait till the BlackCat debugger establishes serial communications with the debug cog.

Starting BlackCat

To start BlackCat, simply enter the following command:

```
blackcat
```

All details (including the program to debug) can be configured in BlackBox once the program has started. See the next section on how to configure and use BlackCat once it has started.

However, **BlackCat** also accepts the following optional parameters:

-P port	Use this to specify the port to use for serial communications – e.g. **COM9**
-B baud_rate	Use this to specify the serial baud rate to use for serial communications – e.g. **115200**
-M mem_model	Use this to specify the memory model – e.g. LMM, CMM, XMMSMALL or XMMLARGE. Note that this is not usually required, as BlackCat will determine this automatically once the port is opened. However, if you do need to specify it, make sure it is specified *after* any port parameter.

-D path_name Use this to specify the path and file name of the **.dbg** file (use double quotes if the path or file name contains spaces).

Here is a complete set of command-line commands that might be used to compile, load and debug a program called *example.c*:

```
catalina -lc example.c -C COMPACT -g3
payload example -p 9
blackcat -P COM9 -B 115200 -D example
```

Using BlackCat

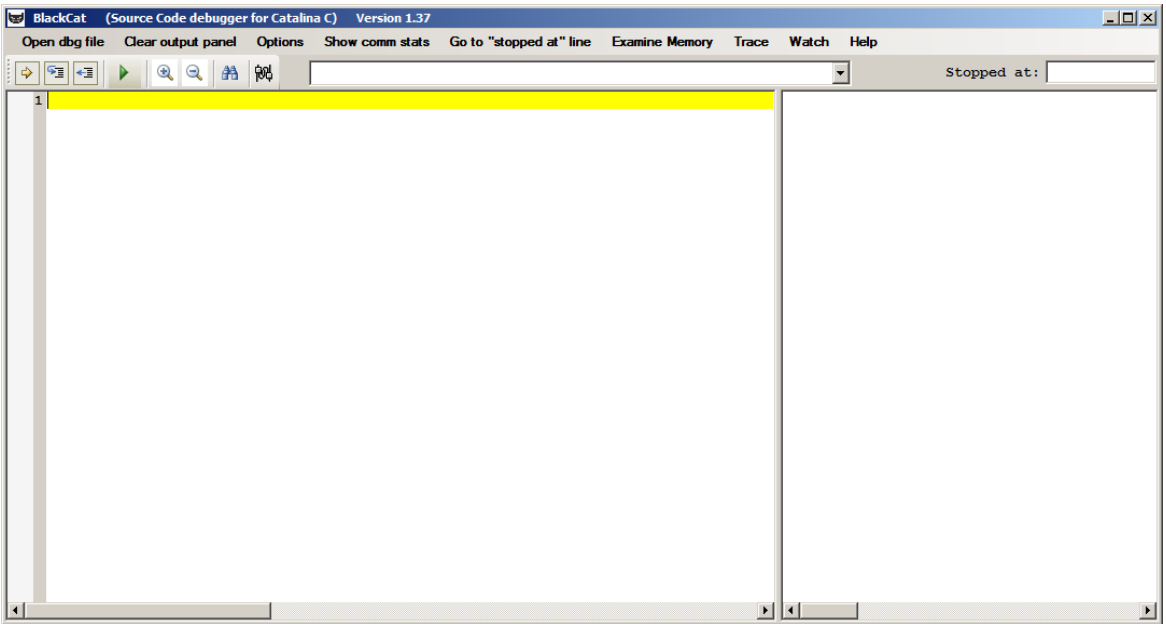
Overview

This section contains a general introduction to each component of BlackCat, such as the windows and commands.

The BlackCat Window

When BlackCat is first started, it may open up to two windows. The larger window is referred to as the BlackCat window. This window is used for displaying source files, setting and clearing breakpoints, viewing and modifying program variables.

It will appear similar to that shown below:



The BlackCat Window contains:

- A menu bar for executing commands
- A tool bar with buttons, a file list dropdown box, and a program counter display. The meaning of the buttons is as follows (their use is described later):

Step (to the next virtual breakpoint);

Step Into (any procedure called on the current line);

Step Out (back to the calling function);

Run to next user breakpoint;

Increase (left click) or decrease (right click) the font size;

Search for text in source files;

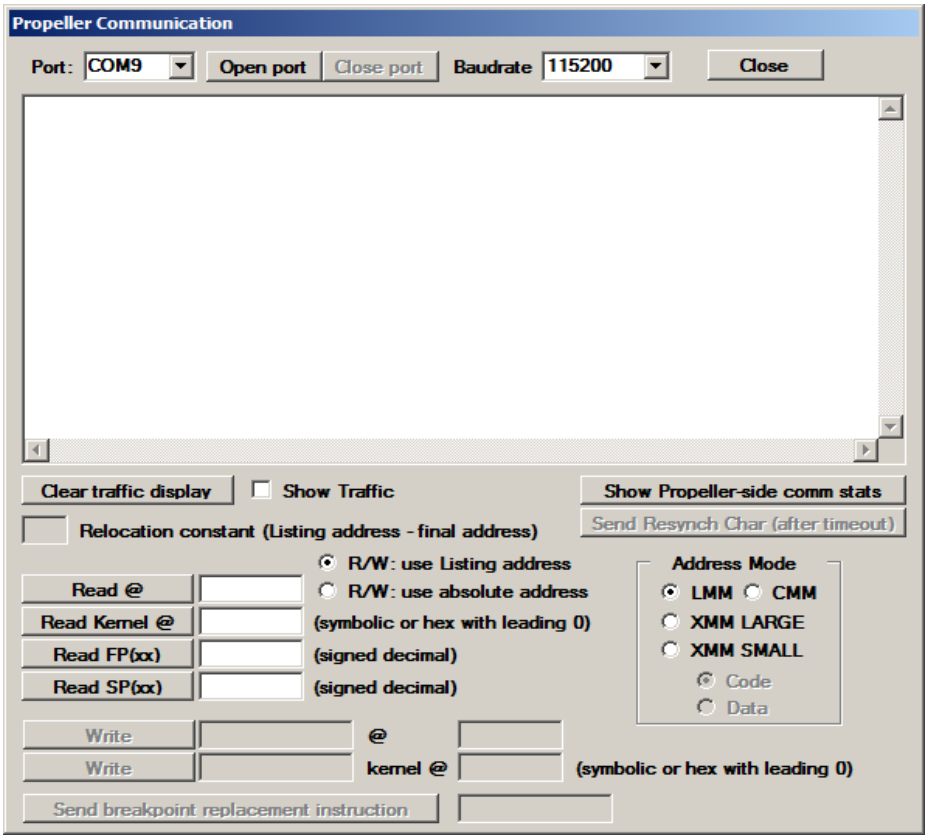
Open (or bring to top) the Propeller Communications window.

- A Source pane. This window is used to display the file selected in the file list dropdown box.
- A Bookmark pane. This window is used to display a list of lines that have been "marked" for future attention. It allows easy navigation back to the line in the future. The bookmark pane will be initially closed.
- An Output pane. This is where all status information is displayed, and where variables visible at each breakpoint are listed.

The Propeller Communications Window

The smaller window that may open when BlackCat is started is referred to as the Propeller Communications window. It will open if BlackBox is not already connected to the Propeller. It will not open if the correct baud rate and port have already been specified on the command line, but it can be opened manually by pressing . It can be left open while the program is being debugged, or closed and then re-opened later should it be required.

When open, it will appear similar to that shown below:



This window contains controls for:

- Selecting a serial port and baud rate, and opening or closing the serial port.
- Closing the Propeller Communications Window (it can be re-opened at any time from the main BlackCat window).

- Viewing communications traffic between BlackCat and the Propeller program.
- Displaying Kernel and Hub or XXM RAM values.
- Selecting whether to use Listing or Absolute addresses. Listing addresses are those used in the leftmost column of the listing file produced by Catalina – but you may notice that these are not the actual addresses used within the instructions – those are absolute addresses.
- Selecting the Address Mode: LMM, CMM, XMM SMALL or XMM LARGE. It is important to do this before a dbg file is loaded, and the value must match the option specified when the program was compiled.
- Modifying Kernel and Hub or XXM RAM values (these controls are disabled by default).

Connecting to the Propeller

The first thing to do once BlackCat is started is to select the serial port and baud rate (if these have not already been specified on the command line).

Both USB and RS232 ports can be used. A baud rate of 115200 should be specified (the program defaults to this). While the baud rate can go up to 921600 but this requires changes to be made to the compiled program (i.e. by manually editing **Catalina_Common.spin** file).

A baud rate of 115200 was chosen as the default as it is appropriate for both USB and RS232 ports. Higher speeds may work on some platforms but not others.

Once these are selected, press the **Open port** button. An error message will be displayed if the port cannot be opened. If the port is opened successfully, there is no message – but you can check if BlackCat is able to communicate with the debug cog running on the Propeller by pressing the **Show Propeller-side Comms Stats** button.

Once the **Com port** and **Baud Rate** are selected, they are remembered from sessions to session. They can also be specified on the command line.

Optionally, select the Address Mode

When the port is opened, BlackCat will automatically determine the memory mode used by the program loaded. About the only reason you would ever need to override the memory mode is if the program to be debugged has not actually been loaded yet when BlackCat is started – but it is generally advisable to load the program first, and then let BlackBox determine the mode automatically.

The mode affects the way memory addresses are interpreted by the debugger, so if you do override it, specifying the wrong mode will lead to the debugger behaving incorrectly.

This setting can also be specified on the command line.

BlackCat Menu Bar Commands

Open dbg file

You need to tell BlackCat to load a dbg file, which must match the binary currently being executed on the Propeller. If the two do not match, the results will be unpredictable.

Since BlackCat immediately starts setting up Virtual Breakpoints once a dbg file is loaded, the Propeller Communications Window must be used to configure and open the serial port, and select the address mode, before a dbg file is loaded. This is why the Propeller Communications window is always opened on top of the BlackCat window. However, once this task is done the Propeller Communications Window can often be closed for the remainder of the debug session.

Before a **.dbg** file is loaded, about all you can do is read specific locations from the kernel or from memory using an absolute address – using listing addresses instead of absolute addresses is not be valid because it requires information in the **.dbg** file (this information is displayed in the **Relocation Constant (Listing address – final address)** field in the Propeller Communications window once a **.dbg** file has been loaded.

Clear Output Panel

This command simply clears the contents of the output panel.

Options

The options menu contains the following additional commands:

Show Virtual Breakpoints

This option (selected by default) determines whether virtual breakpoints will be shown in the left margin of the source window. This is handy as it tells you those lines that have had code generated for them (and hence can hav a breakpoint added) and those that didn't.

Show Traffic in Communications Window

This option causes messages to be logged for all commands between BlackCat and the debug cog visible in the Propeller Communications window. It overrides the Show Traffic option in that window (which does a similar thing).

Remove all Breakpoints

Remove all user and virtual breakpoints that BlackCat has inserted into the program.

Show Comms Stats

Print current BlackCat communications statistics in the output window. Clear the counters.

Go to “stopped at” line

This will cause the line that the Propeller is currently stopped at to be shown in the source pane. The line will be highlighted in red.

Examine Memory

Open a dialog box that can be used to display blocks of RAM.

Trace

Display the trace (call stack) that shows how the program got to the current line.

Watch

Open a dialog box that can be used to a continually updated display of certain variables.

Help

Open a dialog box that displays help for the program.

Displaying files in the Source Window

Once a dbg file is loaded, the source file drop down list will be populated with the names of all the C source files used in the compilation. However, the source file containing the **main()** function will be displayed, and the program will be stopped at the entry point to **main()**.

The left hand column of the source window shows whether or not there are any virtual or user breakpoints that refer to the line, and also whether the program is stopped on the displayed line:

Virtual Breakpoint

User Breakpoint

Right clicking in this window brings up the following menu:

Bookmark highlighted line adds the highlighted line to the lines listed in the bookmark window. **Remove all breakpoints from all files** is similar to the **Remove all Breakpoints** menu item.

Using Bookmarks

Right-clicking the mouse on a line in the bookmark window brings up the following menu:

Go to { line } … displays the highlighted line in the source window. **Delete highlighted bookmark** removes a bookmark, and **Clear bookmarks** removes all bookmarks from the current file.

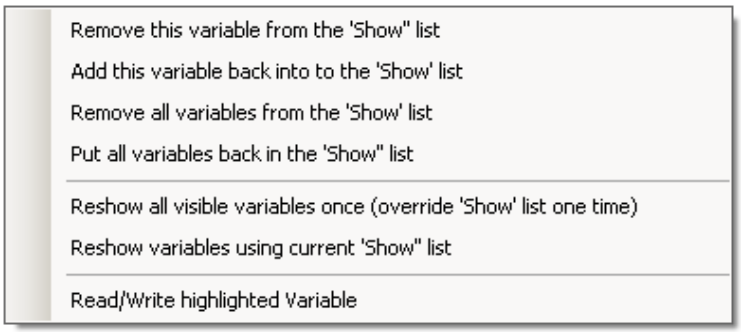
Bookmarks are saved from session to session. Only the bookmarks for the current file are shown.

Managing the Variable "Show" list or Read/Write Variables

The "Show" list is the list of all variables – global or local that are visible from the breakpoint at which BlackCat is currently stopped. It is displayed in the output window after each and every breakpoint (user or virtual).

If the list of such variables is large, this can be slow, and make it difficult to locate a value of particular interest.

So BlackCat provides a means of managing the "Show" list. Right clicking on a highlighted line in the output window brings up the following menu:



Remove this variable from the 'Show' List – remove the highlighted variable from the list of those shown at each breakpoint.

Add his variable back into the 'Show' list – show the highlighted variable at each breakpoint.

Remove all variables from the 'Show' list – do not show any variables at each breakpoint.

Put all variables back in the 'Show' list – show all variables at each breakpoint again.

Reshow all visible variables once (override 'Show' list one time) – show all variables once – useful to find variables that may need to be added back into the show list.

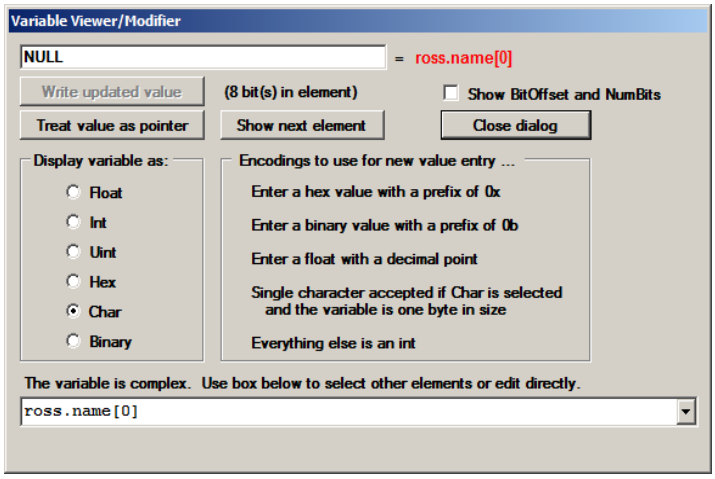
Reshow variables using current 'show' list – redisplay the current list – useful if there are variables that may be updated by another executing cog.

Read/Write highlighted Variable – open a dialog box that allows the value of the highlighted variable to be displayed in more detail, and/or modified (see next section).

Displaying and Editing Variables

When the "Show" list is displayed in the output window, highlighting a line containing a variable (by left clicking on it) and then right clicking to bring up the menu and

selecting Read/Write highlighted variable displays the Variable Viewer/Modifier dialog box:



This box contains the name of the variable, its location (register, frame or global), it's current value and size. Also, if the variable is not a simple scalar (e.g. it is a structure, union or array) the dropdown list will enumerate all the possible scalars that comprise the object. A new value can be written to each scalar, or scalar component of a more complex object, and if the variable contains a pointer it can be de-referenced to display the value that the pointer currently points to (and dereferenced again, if that value is also a pointer). The formatting used to display the scalar value is selectable.

An example of using BlackCat

Overview

This section provides a real-world example of using BlackCat. It uses an example program provided with the Catalina distribution, in the *demos\debug* directory.

The example program requires an external VGA or TV display and keyboard, but no mouse or floating point libraries.

Compile the example program

The example program provided consists of the following C source files:

```
debug_test.h
debug_main.c
debug_functions_1.c
debug_functions_2.c
```

A batch file called **build_all.bat** is provided to build this example program (a **Code::Blocks** project is also provided, which by default builds for the **CUSTOM** platform – you may need to edit this for your platform).

To build the example program, go to the *demos\debug* directory and run the batch file, specifying your Propeller platform as a command line parameter. If that platform has multiple CPUs, also specify the CPU.

For example:

```
build_all TRIBLADEPROP CPU_1 NO_MOUSE
```

or

```
build_all HYDRA COMPACT NO_MOUSE
```

This will generate the following files:

```
example.binary
example.dbg
example.lst
debug_main.debug
debug_functions1.debug
debug_functions2.debug
```

Note that if you use the batch file to compile the example, the binary (and corresponding **.lst** and **.dbg** files) will be located in the *demos\debug* directory, whereas if you use **Code::Blocks** to compile the example, the files will be located in the *codeblocks\debug_example\bin\Debug* directory.

Loading the example program

Load the **example.binary** file into the Propeller using the Parallax Propeller Tool. For instance:

```
payload example
```

When the binary has been loaded, all the drivers are initialized, so you should see a blank display appear. However, the C program will not begin to execute until we start BlackCat.

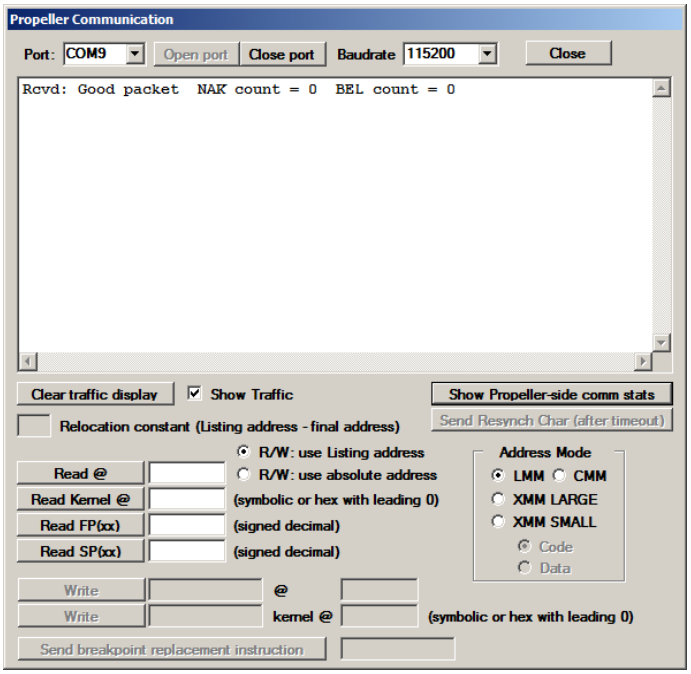
Starting BlackCat, establishing comms and loading the dbg file

Start BlackCat using the command:

```
blackcat
```

When the BlackCat and Propeller Communications windows open, select the serial port and baud rate (use 115200 baud). Then press **Open port**.

There is no message, but you can manually verify the serial communications is working by selecting **Show Traffic** and then pressing **Show Propeller-side comm stats**. Here is what you might see if you had your Propeller connected to COM9:

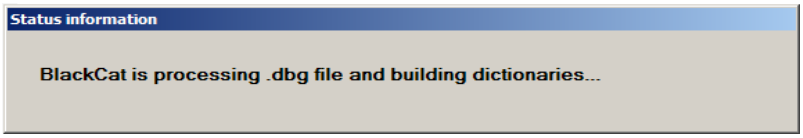


You can now close the Propeller Communications window (press the **Close** button at the top right). If we need to re-open this window later, we can do so from the main BlackCat window.

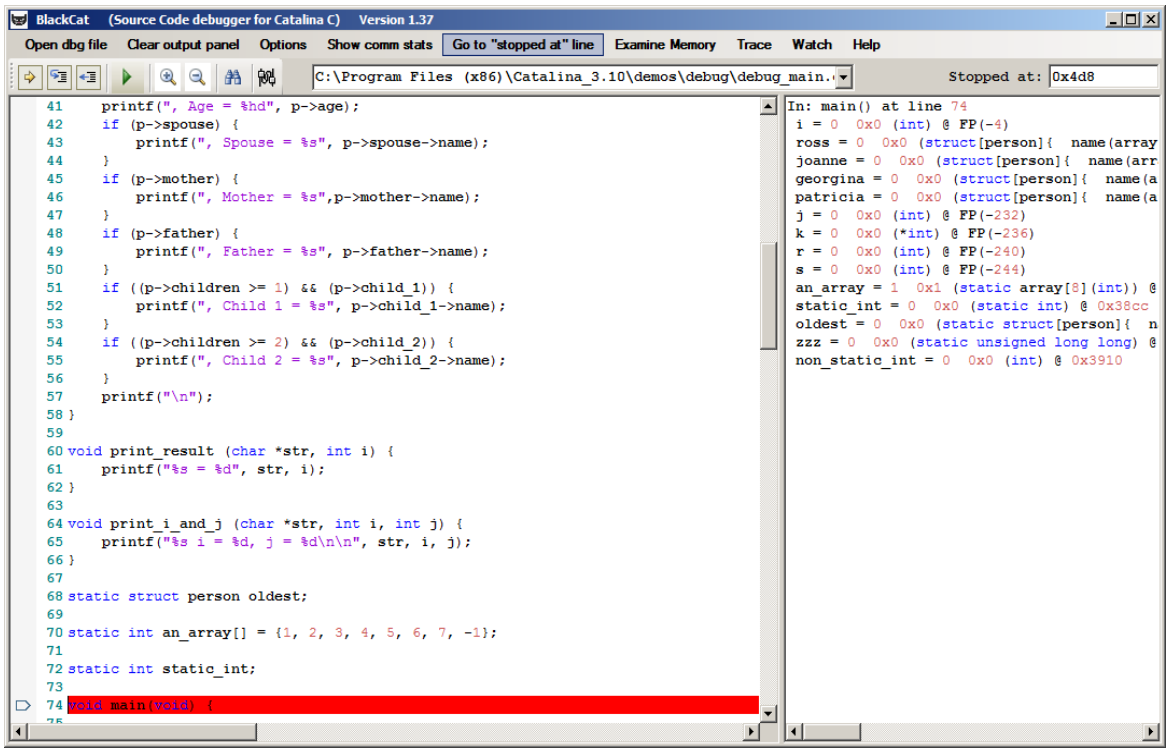
The Propeller Communications window can be opened by pressing the button.

In the BlackCat main window, Press **Open dbg file**. A dialog box will appear in which you can select the **.dbg** file to load – it must be one that corresponds to the binary you have loaded into the Propeller.

A message will appear similar to the following while the dbg file is being loaded and processed:



One the load is complete (it may take a few seconds) the example program should appear in the BlackCat source window, similar to that shown below:



Note that the mark in the margin, and the red highlighting, shows the program has stopped at a virtual breakpoint that has been automatically placed on the beginning of the **main** function.

Note also that the progress indicator (top right) indicates the address the program is stopped at (**0x4d8**), and the output window contains a message about where the program is stopped (**in main() at line 74**) and also a printout of all variables visible at this point.

Scroll within the source window to display more of the main function. You will notice markers for virtual breakpoints that have been set at every executable line in the **main** function. These virtual breakpoints are used to step through the program line

by line, and show where the program will stop at each step (since not all lines have code generated for them).

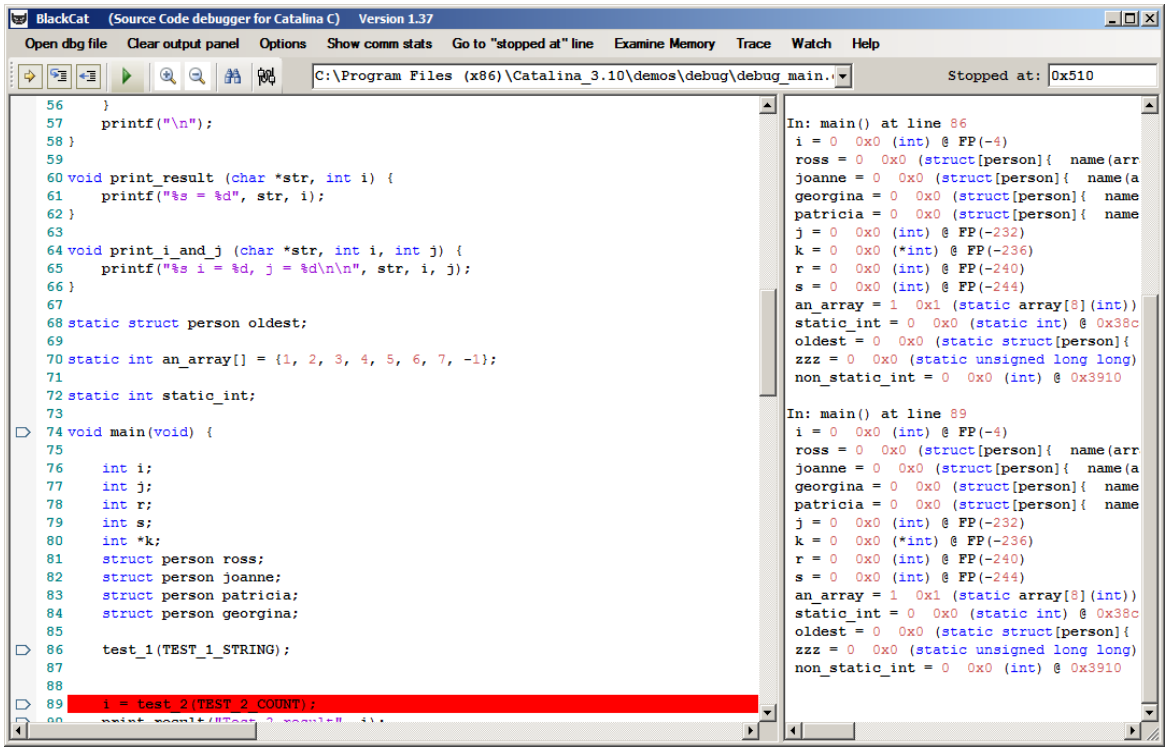
Using Step, Step Into & Step Out

To execute a single 'Step', you press the button. Do this now. The program should step to line 86, the variable list will be displayed in the output window, and the source window will appear.

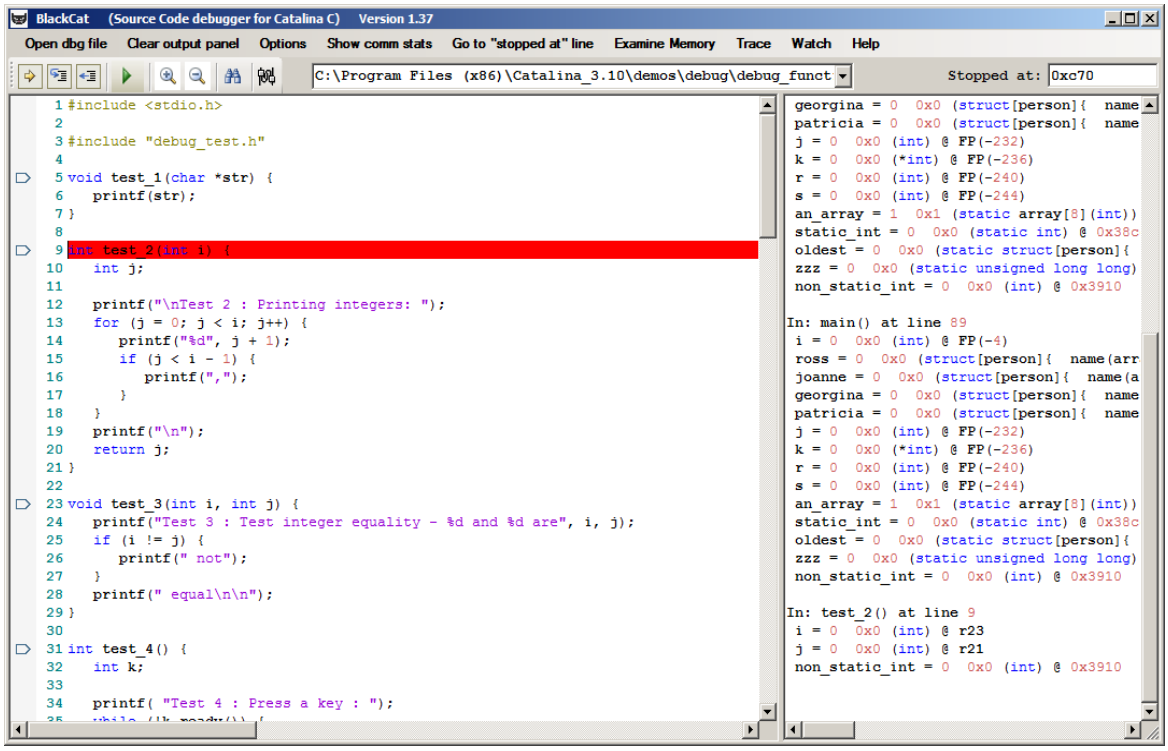
Press 'Step' one more time to step to line 89.

The display attached to the Propeller should now show a line of text on it (put there by the function call executed on line 86).

The BlackCat window should now appear similar to the following:



This time, instead of pressing 'Step' (which steps over function calls), press to 'Step Into' those calls. The program will step into the function **test_2**.

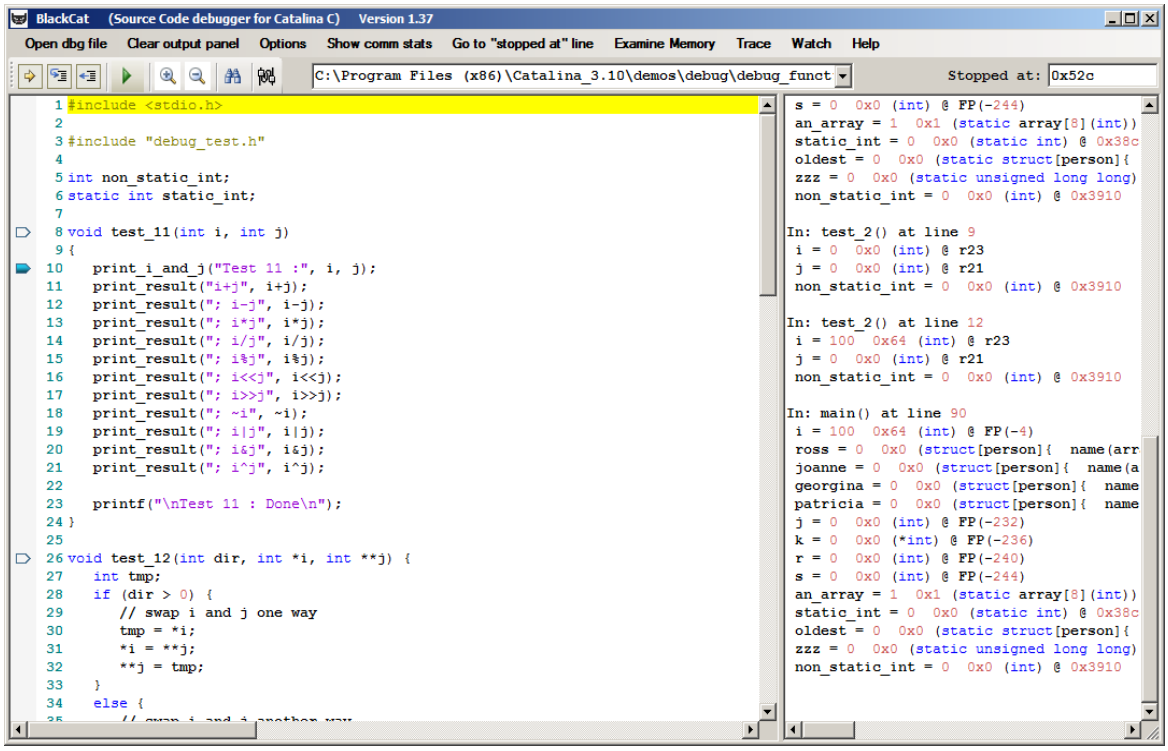


Now press ‘Step’ to step to line 12. Notice how the virtual breakpoints change as you do so.

Then press ‘Step Out’ to return to line 90 – i.e. the line after the procedure we ‘Stepped Into’.

User Breakpoints

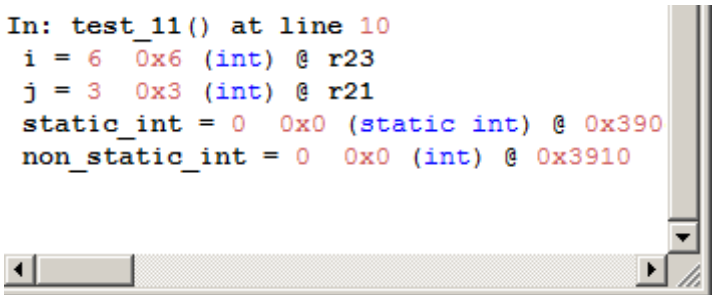
Sometimes, we just want to set a user breakpoint on a specific line and let the program run till it reaches it. In this case, we will now set a breakpoint on line 10 of the file **debug_functions_2.c** (in function **test_11**). We change file by selecting the file from the file drop down list at the top of the file window, and put a user breakpoint by clicking next to line 10 in the left hand margin. The result will look something like this:



Now we simply want to let the program continue till it reaches this user breakpoint.

To do that we use the 'Run' button. This button will cause the program to execute till it reaches the user breakpoint – it will not stop at virtual breakpoints on the way. Press it now.

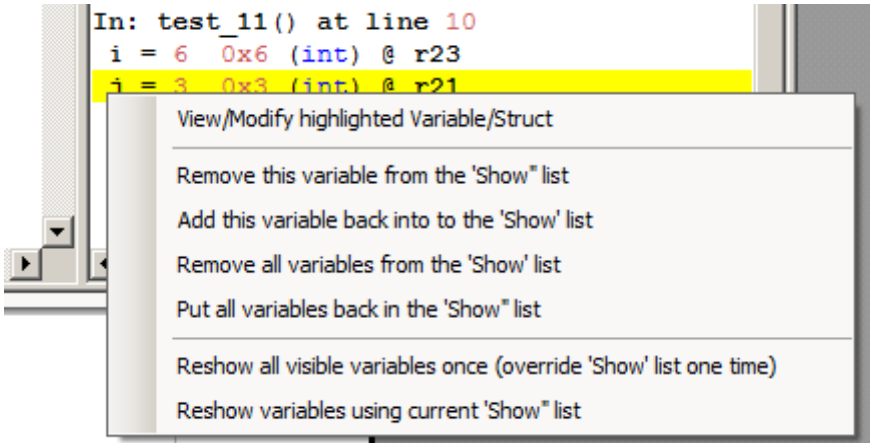
There may be a pause while the program removes all the virtual breakpoints it has previously placed (to support the single stepping we did earlier), then the program will proceed. Note also that the program is interactive – you need to press enter on the keyboard a couple of times. Then the program will stop at line 10. Note that the output panel displays the values of the variables **i** and **j**:



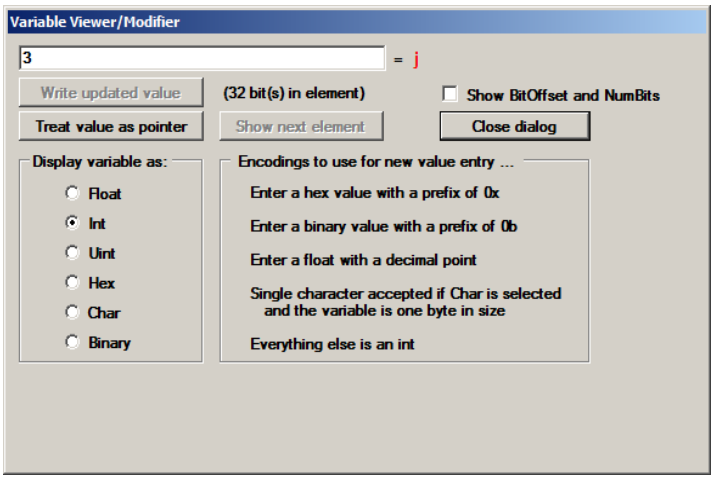
We will now modify one of these variables and then let the program continue.

Viewing and Modifying Variables

To view or modify a variable, left click to highlight the line containing the variable in the output panel (in yellow), then right click to bring up the menu and select **Read/Write highlighted Variable**.



A dialog box similar to the following will appear:



This dialog box can be used to display the scalar value of the variable (as a **float**, an **int**, an **unsigned int** or a **hex** value), to de-reference it (if the variable in fact contains a pointer to a scalar data type rather than the data type itself) by pressing the **Treat value as pointer button**. It can also be used to change the current value of the variable.

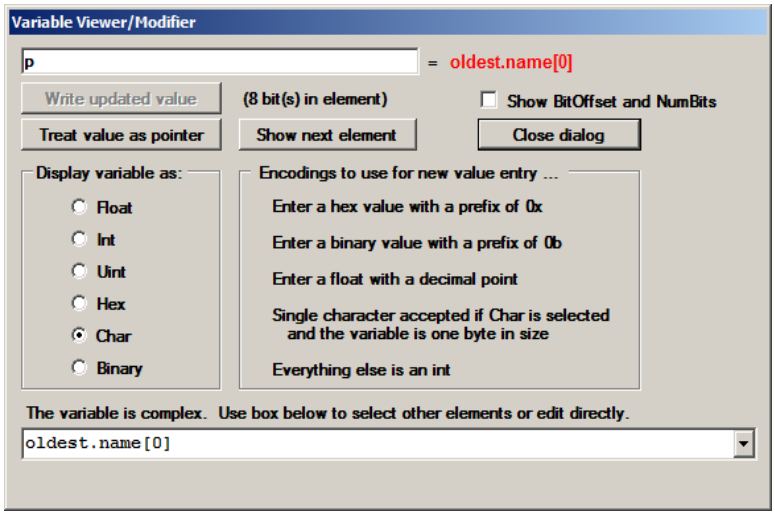
You can change the value by typing the new value in the value field, then pressing the **Write updated value** button, then **Close Dialog** to close the dialog box. Try this now – change the value of **j** to 4.

If we 'Step' the program, we will see the new value in use.

If the value is complex (an array or structure) a dropdown list appears that allows the selection of the element of an array, or the member of a structure. For a simple scalar variable (such as **j**), the dropdown list does not appear To see it in use for a non-scalar variable, remove the user breakpoint from line 10 of the current file, and select the file **debug_main.c.** Put a user breakpoint on line 161 in function **main**, just after the calls to **test_11**. Then press 'Run' (if you didn't remove the user breakpoint in function **test_11**, you will need to press 'Run' more than once since that function is called twice).

Then highlight the line in the Output pane containing the variable **oldest** and open the Read/Modify dialog box again.

This time, the drop down list (at the bottom of the dialog box) contains a list of all the elements of the **oldest** structure, including the element **name**, which is an array:



In addition to selecting from the list, you can also type in the name of the structure member or array element and press TAB. If the name is valid it will be displayed.

This is the end of the example. You should continue single stepping through the example, displaying variables etc, until you are familiar with all the basic BlackCat functionality.

Known Issues with BlackCat

This release of BlackCat has a few limitations and known issues:

- The source window cannot be used to display **.h** files.
- BlackCat cannot resolve symbols that have been redefined using **#define**.
- The length of the file selection dropdown list is too short too display the full filename in some cases.

Advanced Topics

Debug files and listing files

The debug files contain stabs format information generated by the LCC compiler. This is the raw information that is used to build the dbg file that BlackCat uses.

The listing files are generated by Homespun, and are useful for determining program code and data addresses for use in BlackCat. Note that the addresses given on the listing are not absolute addresses – they mainly represent where the object ended up in the binary file generated by Homespun, and not where the object would be physically loaded at run time. However, once a dbg file has been loaded into BlackCat, BlackCat can do the necessary conversion to allow either listing or absolute addresses to be used.

Modifying Kernel and Hub / XMM memory locations

To enable the normally disabled functions in the Propeller Communications Window to allow kernel and hub memory locations to be modified directly, enter the word

toggle in the **Read Kernel @** data entry box, and then press the **Read Kernel @** button.

Note that this can only be done while BlackCat is stopped at a breakpoint.

Modifying or Recompiling programs

To prevent erroneous results, if BlackCat detects that the source files, the debug files, the listing files or the binary files are modified while the program is being debugged, BlackCat will print a message in the output window telling you to shut BlackCat down and restart it. Not doing so when this message appears will lead to unpredictable results.